

Aclaraciones de la reunión 9-6-26

Samuel Corrionero Fernández

Resumen

Este documento explica algunos conceptos que nos hicieron reflexionar en la última reunión. En primer lugar, se detalla cómo se ha normalizado el cálculo del Hipervolumen (HV) para que funcione correctamente según el modo de evaluación. Después, se analizan las diferencias entre usar el modo proporcional de Ting *et al.* frente a un modo absoluto, y se justifica por qué el objetivo de tolerancia (f_2) debe mantenerse siempre en valores absolutos para evitar que el algoritmo colapse. Finalmente, se incluye una revisión del código fuente original en C++ para confirmar que su implementación coincide realmente con las fórmulas teóricas del *paper*.

Índice

1. Hipervolumen según el modo de evaluación	2
1.1. Normalización del Hipervolumen según el modo de evaluación (absoluto o proporcional)	2
1.2. Diferencias en los resultados del hipervolumen según el modo de evaluación	3
2. Análisis del Modo de Proporcionalidad de Ting	4
2.1. La división de f_3 y f_4 entre k : De Fuerza Bruta a Eficiencia	4
2.2. Por qué NO se divide f_2 entre k : El Equilibrio del Motor Evolutivo	5
2.3. La falsa redundancia: Transformación semántica vs. Dependencia lineal	6
2.4. Desglose de las Fórmulas (Paper vs. Código de Ting)	7

1. Hipervolumen según el modo de evaluación

1.1. Normalización del Hipervolumen según el modo de evaluación (absoluto o proporcional)

En la última reunión surgió la duda de si la implementación del Hipervolumen (HV) podría contener algún error matemático, ya que daba la sensación de que el cálculo no se adaptaba correctamente al modo de evaluación. Sin embargo, tras revisar a fondo el código, he confirmado que la implementación del cálculo es totalmente correcta.

En realidad, la gran diferencia de valores entre las ejecuciones anteriores y las actuales se debe a un detalle en mi antigua implementación del modo proporcional: había modificado la fórmula original para que el objetivo de tolerancia (f_2) también se dividiese entre el tamaño del subconjunto (k). Al deshacer este cambio y restaurar el modo proporcional estricto de Ting *et al.* (donde la tolerancia se mantiene en valores absolutos), el algoritmo ha dejado de colapsar y los resultados del hipervolumen han vuelto a ser coherentes.

A continuación, explico detalladamente cómo funciona la normalización dinámica del Hipervolumen por si le quieres echar un vistazo al razonamiento matemático.

Para que el Hipervolumen (HV) proporcione una métrica comparativa justa y no sesgada en un entorno multiobjetivo, es un prerequisite matemático estricto que todos los objetivos dimensionales estén normalizados a la misma escala (típicamente el intervalo unitario $[0, 1]$).

El motor desarrollado garantiza esta normalización perfecta aplicando la fórmula clásica de escalado *Min-Max* antes de someter el frente de Pareto a la evaluación de PyMoo. Para ello, se calculan dinámicamente dos vectores límite: el punto *ideal* (el mejor valor posible o mínimo algorítmico) y el punto *nadir* (el peor valor posible o máximo algorítmico).

La robustez de la implementación reside en la sobrescritura dinámica de las cotas máximas en función del modo de evaluación (absoluto o proporcional). La lógica estructural se define en el siguiente bloque simplificado del núcleo `metrics_logic.py`:

```
1 # 1. Por defecto, los limites asumen el espacio poblacional absoluto
2 max_ham = L # La distancia media nunca superara el total de SNPs
3 max_var = (L**2)/4 # Varianza maxima matematica para L elementos
4
5 # 2. Si se activa el modo proporcional, los objetivos 3 y 4 se evaluan
6 # como ratios o porcentajes, sobrescribiendo sus cotas teoricas:
7 if modo_evaluacion == 'proporcional':
8     max_ham = 1.0 # La distancia media proporcional maxima es 100% (1.0)
9     max_var = 0.25 # La varianza maxima de cualquier variable acotada en [0, 1]
10
11 # 3. Se construyen los vectores que delimitaran el hipercubo unitario [0, 1]
12 # Nota: El punto 'ideal' de las distancias se inyecta en negativo (-max)
13 # porque el codigo transforma previamente las maximizaciones en minimizaciones.
14 ideal = np.array([1.0, -max_tol, -max_ham, 0.0]) # Minimios algoritmicos
15 nadir = np.array([L, 0.0, 0.0, max_var]) # Maximos algoritmicos
```

El diseño matemático de esta transformación garantiza una evaluación fidedigna para ambos paradigmas gracias a la gestión paramétrica de los extremos:

- **1. Comportamiento bajo el Modo Absoluto:** En este escenario, las distancias dependen íntegramente del número total de SNPs polimórficos del dataset (L). Al no ejecutarse el bloque condicional proporcional, el algoritmo inyecta los máximos estadísticos poblacionales absolutos en la formulación:
 - **Para f_3 (Distancia media):** Su mejor valor teórico (el *ideal*) pasa a ser `-max_ham` (el extremo negativo de la distancia máxima) y su *nadir* es 0,0. La escala resultante acota el denominador estrictamente a la amplitud absoluta de la distancia.

- **Para f_4 (Varianza):** Dado que estadísticamente la varianza de poblaciones idénticas es invariablemente 0,0, el punto **ideal** queda anclado universalmente en este valor, mientras que el **nadir** recoge el límite teórico absoluto $L^2/4$.
- **2. Adaptación bajo el Modo Proporcional:** En este paradigma, f_3 se divide por el tamaño del subset (k), convirtiéndose en un ratio de eficiencia, y f_4 se divide por k^2 . Mantener los límites del escenario anterior pulverizaría los valores del hipervolumen acercándolos asintóticamente a cero. Sin embargo, la sobrescritura dinámica de las variables corrige este desfase geométrico:
 - **Para f_3 (Eficiencia media):** La proporción máxima de distancia teórica es exactamente 1,0. Al actualizarse la variable `max_ham` = 1.0, el punto **ideal** se recalcula de forma transparente a -1,0. Al restar el **nadir** (0,0), el denominador escala matemáticamente a 1,0.
 - **Para f_4 (Varianza proporcional):** El límite inferior (**ideal**) se mantiene universal en 0,0. Por el teorema del límite de la varianza en variables acotadas en $[0,1]$, la máxima varianza posible es estadísticamente 0,25. Al actualizarse `max_var` = 0.25, la escala se ajusta automáticamente a $[0,0,0,25]$ y el denominador resultante es 0,25.
- **3. El cálculo del Hipervolumen y la flexibilidad del Punto de Referencia:** Una vez que el frente original ha sido normalizado matemáticamente al intervalo unitario usando los vectores descritos, el motor procede a calcular el volumen.

```

1 # Funcion delegada a PyMoo para calcular el area dominada
2 punto_ref = np.ones(n_objetivos) * 1.1
3 return HV(ref_point=punto_ref).do(frente_normalizado)

```

Podría parecer contradictorio que el punto de referencia se fije estáticamente en 1.1 (representando el vector $[1.1, 1.1, 1.1, 1.1]$) independientemente de si evaluamos proporciones o distancias absolutas brutas. Sin embargo, **es precisamente este anclaje geométrico estático el que demuestra la flexibilidad paramétrica del modelo.**

El indicador `HV(ref_point=1.1)` opera *exclusivamente sobre el espacio post-normalizado*. Como se ha demostrado en las secciones anteriores, la función encargada de los límites ya ha absorbido y abstraído toda la variabilidad de escala (ya fuera $L^2/4$ o 0,25). Al recibir la matriz, el cálculo de Hipervolumen desconoce la magnitud original de los datos; simplemente asume matemáticamente que todos los frentes de Pareto están uniformemente empaquetados en un hipercubo de $[0,1]^4$.

Un punto de referencia de 1.1 significa, universalmente, *“calcular el volumen delimitado entre la solución y un punto geométrico un 10 % peor que el máximo Nadir normalizado”*. Esta separación de contingencia del 10 % garantiza que las soluciones más extremas que llegasen a colisionar con el límite Nadir sigan aportando diferencial de volumen sin arrojar un área matemática nula.

Conclusión: Gracias a este modelo de sustitución escalar, la arquitectura asegura que el cálculo del área no sufra sesgos ni deformaciones topológicas. La evaluación devuelta por PyMoo usando el punto de referencia 1.1 está aislada de la semántica original de los datos y depende estrictamente de cómo el código ha modelado de forma impecable los límites del frente, resultando en una métrica algorítmicamente rigurosa e inalterable ante los cambios de configuración.

1.2. Diferencias en los resultados del hipervolumen según el modo de evaluación

Como bien observaste en revisiones anteriores, las primeras pruebas del motor arrojaban valores de Hipervolumen (HV) extremadamente pequeños cuando se activaba el modo proporcional, lo cual generaba dudas razonables sobre la integridad de la métrica. Tras una revisión exhaustiva de la formulación matemática y revisando la literatura base, he podido aislar el origen de esta caída de magnitudes.

El problema radicaba en que mi implementación original aplicaba una “proporcionalidad extrema”, forzando a que absolutamente todos los objetivos (incluida la tolerancia, f_2) fuesen divididos por el tamaño del subconjunto (k o f_1). Como me indicaste, para hacer el estudio comparable con el de

Ting *et al.*, he retornado la formulación a su forma original, donde la distancia y varianza se hacen proporcionales, pero la tolerancia (f_2) se mantiene como un umbral en valores absolutos. Es decir, f_2 ya no se divide entre f_1 .

Para ilustrar el impacto de esta corrección, la siguiente tabla muestra la evolución empírica del hipervolumen tomando como referencia el algoritmo NSGA-II (con inicialización *Random Dense*):

Modo de Evaluación	Tolerancia (f_2)	Distancia (f_3)	HV
Absoluto	Cruda (Absoluta)	Cruda (Absoluta)	$\approx 0,536$
Proporcional (Ting)	Cruda (Absoluta)	Eficiencia (Dividida por k)	$\approx 0,400$
Proporcional Extremo (Anterior)	Porcentaje (Dividida por k)	Eficiencia (Dividida por k)	$\approx 0,011$

Cuadro 1: Evolución empírica del hipervolumen en NSGA-II según el modo de evaluación.

El análisis de esta tabla revela tres realidades topológicas distintas sobre cómo el algoritmo navega el espacio de búsqueda:

- **1. El colapso del Proporcional Extremo (Anterior):** Al dividir la tolerancia mínima entre k , se castigaba el tamaño de la solución por partida doble (tanto en f_3 como en f_2). Matemáticamente, la división comprimía soluciones inmensamente diferentes en valores decimales minúsculos casi idénticos. Esto generaba un aplastamiento severo del frente de Pareto, hundiendo el hipervolumen a niveles residuales (0,011). Fue tu observación la que permitió detectar que esta agresiva modificación exploratoria desvirtuaba el espacio de soluciones.
- **2. La estabilización en el Proporcional de Ting (Actual):** Al retornar la tolerancia a valores absolutos (tal y como Ting lo definió originalmente), se elimina el “aplastamiento matemático”. Las soluciones ahora pueden separarse de forma natural en el eje de la tolerancia, lo que permite que el frente respire y el hipervolumen recupere su estabilidad geométrica subiendo a 0,400.
- **3. La diferencia biológica con el Modo Absoluto:** Incluso tras estabilizar la métrica, se observa que el modo de Ting arroja un volumen menor (0,400) que el modo Absoluto (0,536). Esta pérdida del $\sim 25\%$ no es producto de la penalización matemática extrema que probé en la fase anterior, sino de una consecuencia directa de la biología del problema. En el Modo Absoluto, el algoritmo permite el crecimiento descontrolado de SNPs, acumulando distancia bruta y extendiendo el frente a lo largo de todo el espacio de búsqueda. En el Modo Ting, el algoritmo exige mantener una alta *eficiencia por SNP*. Como mantener una eficiencia cercana al 100% es biológicamente imposible a medida que la solución crece, el algoritmo poda drásticamente la fuerza bruta, dando lugar a un frente mucho más selecto y compacto que, de forma completamente natural, cubre un volumen matemático menor.

2. Análisis del Modo de Proporcionalidad de Ting

2.1. La división de f_3 y f_4 entre k : De Fuerza Bruta a Eficiencia

En la formulación original de Ting *et al.*, se establece de forma explícita que “la similitud de los contextos haplotípicos se mide mediante la distancia Hamming en los Tag SNPs seleccionados”. Al trasladar este concepto biológico a un motor evolutivo multiobjetivo, surge una necesidad ineludible: transformar la distancia absoluta (fuerza bruta) en una medida relativa (eficiencia marginal).

El problema de la fuerza bruta (Distancia Cruda)

Si operamos con la distancia media sin dividir (f_3 cruda), generamos una competición desleal en el espacio de búsqueda. Por pura estadística poblacional, cada SNP que se añade al panel garantiza un incremento lineal en la distancia total, independientemente de su calidad. Esto ciega a las reglas de dominancia de Pareto. Al analizar los datos empíricos de nuestras ejecuciones en **Modo Absoluto**, observamos que la recompensa por engordar el subconjunto es masiva:

- Con $k \approx 20$, se obtiene una distancia cruda de $f_3 = 15,0$
- Con $k \approx 100$, la distancia sube a $f_3 = 46,4$
- Con $k \approx 500$, la distancia se dispara a $f_3 = 193,6$

Ante un incremento asegurado de casi 180 puntos de distancia bruta, el algoritmo evolutivo decide ignorar el coste de añadir SNPs (objetivo f_1). La recompensa en f_3 es tan desproporcionada que genera *trade-offs* matemáticamente válidos pero biológicamente inútiles, estirando el frente de Pareto hacia paneles gigantes de hasta 500 SNPs repletos de mutaciones redundantes.

La correa de la Eficiencia (f_3/k)

Al dividir la distancia total entre k , Ting transforma la métrica: el algoritmo ya no premia la cantidad total de mutaciones, sino el **poder de discriminación promedio que aporta cada SNP de forma individual**.

Los datos empíricos extraídos de las ejecuciones en **Modo Proporcional** demuestran el éxito radical de esta corrección:

- Con $k < 20$, la eficiencia es de $f_3/k = 0,46$
- Con $k \approx 100$, la eficiencia cae a $f_3/k = 0,40$
- Con $k \approx 440$, la eficiencia se desploma a $f_3/k = 0,35$

Al dividir entre k , la ganancia ya no está asegurada. Al contrario, la eficiencia **disminuye estrictamente** a medida que se satura el panel. Esto genera un auténtico conflicto multiobjetivo: el algoritmo ya no puede crecer impunemente porque cada SNP ineficiente que añade hunde su puntuación en f_3 . Como resultado, la división por k actúa como una guillotina que purga los paneles hinchados, premiando únicamente a los subconjuntos pequeños y altamente informativos, cosa que en el modo absoluto no ocurría. Ahora, f_3 en vez de premiar la robustez (muchos SNPs) premia la compacidad.

La Varianza Proporcional (f_4/k^2)

Este mismo cambio de paradigma ocurre con la estabilidad clínica. En el Modo Absoluto, la varianza cruda explotaba linealmente al añadir SNPs (nuestros datos muestran que crecía de $\approx 3,88$ a más de 965,23). Como el motor evolutivo busca minimizar este objetivo, f_4 actuaba como un castigo feroz contra el crecimiento, forzando implacablemente la compacidad extrema. Sin embargo, al aplicar la rigurosa división estadística entre k^2 en la formulación de Ting, calculamos la varianza de la eficiencia. Es aquí donde entra en juego la Ley de los Grandes Números: el comportamiento se invierte por completo porque los paneles grandes suavizan las rarezas estadísticas. Empíricamente, la varianza proporcional en nuestras pruebas cayó en picado de 0,0168 (en $k < 20$) a 0,0027 (en $k \approx 440$). Al aplicar la proporcionalidad, f_4 *cambia de bando*: deja de forzar la compacidad y pasa a premiar sutilmente a los paneles grandes por ser estadísticamente más equilibrados y predecibles, añadiendo una valiosa capa de robustez diagnóstica al modelo.

2.2. Por qué NO se divide f_2 entre k : El Equilibrio del Motor Evolutivo

Si hemos establecido que dividir la distancia (f_3) y la varianza (f_4) entre k es beneficioso para medir la eficiencia y la estabilidad por SNP, la pregunta natural es: *¿Por qué no aplicar la misma lógica proporcional a la tolerancia (f_2)?* La respuesta reside en el equilibrio de fuerzas que gobierna el motor de búsqueda multiobjetivo.

En la arquitectura de nuestro problema, las funciones objetivo actúan como fuerzas que “tiran” de las soluciones en direcciones opuestas para generar el frente de Pareto:

- **La fuerza hacia la compacidad (f_1 y f_3 proporcional):** El objetivo f_1 busca explícitamente minimizar el tamaño del subconjunto. Paralelamente, como hemos comprobado empíricamente, la eficiencia marginal (f_3/k) decae a medida que se añaden más SNPs. Por tanto, tanto f_1 como f_3 castigan severamente a los paneles grandes, empujando al algoritmo a buscar soluciones lo más pequeñas posibles.

- **La fuerza hacia la robustez (f_2 absoluta y f_4 proporcional):** La tolerancia cruda (f_2) es la única recompensa matemática directa que tiene el algoritmo para permitirse explorar tamaños mayores. Al añadir SNPs, el algoritmo tiene la oportunidad (aunque lenta) de incrementar el margen de error absoluto permitido. Paralelamente, por la Ley de los Grandes Números, la varianza (f_4) disminuye (mejora) al aumentar k , favoreciendo la estabilidad estadística de los paneles amplios.

El colapso empírico por proporcionalidad total (Modo Extremo)

Si cayéramos en la tentación de hacer f_2 proporcional (dividiendo también el umbral mínimo entre k), destruiríamos este ecosistema. Matemáticamente, una tolerancia de 5 conseguida con $k = 20$ daría una proporción de 0,25, mientras que una tolerancia blindada de 100 conseguida con $k = 500$ daría 0,20. Al convertir f_2 en proporción, esta métrica pasa al bando de la compacidad. Con f_1 , f_2 prop., y f_3 prop. tirando simultáneamente hacia $k \rightarrow 0$, las fuerzas se desequilibran por completo.

No se trata de una hipótesis teórica: **es exactamente lo que ocurrió empíricamente cuando implementé el Modo Proporcional Extremo** en fases anteriores del estudio. Al dividir f_2 entre k , el algoritmo se quedó sin ningún incentivo para explorar soluciones amplias y robustas. El frente de Pareto colapsó gravitatoriamente hacia el origen, incapacitando al motor para generar diversidad y hundiendo la métrica de **Hipervolumen a niveles residuales** ($\approx 0,011$), lo que supone una pérdida de volumen matemático del 97 % respecto al frente estabilizado ($\approx 0,400$).

Por lo tanto, mantener la asimetría de Ting (f_3 proporcional y f_2 absoluta) no es una inconsistencia matemática, sino una necesidad de diseño de ingeniería algorítmica indispensable para mantener la tensión multiobjetivo, evitar el colapso del frente y encontrar un *trade-off* genuino.

2.3. La falsa redundancia: Transformación semántica vs. Dependencia lineal

Durante las revisiones previas se planteó una objeción teórica muy pertinente: el principio de independencia en la optimización multiobjetivo. Según este principio clásico, los objetivos deben ser ortogonales y no estar correlacionados. Bajo esta premisa, incluir el tamaño del subconjunto ($f_1 = k$) como denominador dentro de las funciones de distancia (f_3) y varianza (f_4) podría interpretarse como una redundancia que viola la independencia del modelo.

Si bien este principio es innegable en la teoría pura, la formulación de Ting no incurre en una dependencia lineal, sino que realiza una **transformación semántica** indispensable para la viabilidad computacional del problema.

1. Ausencia de correlación estricta

Una verdadera dependencia matemática ocurriría si el segundo objetivo fuese una función directa del primero (por ejemplo, $f_3 = 2k + 5$). En ese escenario, el frente de Pareto degeneraría porque optimizar un objetivo automáticamente optimizaría el otro. Sin embargo, la ecuación de Ting es $f_3 = \bar{D}(S)/k$. El valor bruto de la distancia ($\bar{D}(S)$) no depende exclusivamente de k , sino de la **calidad biológica** de los SNPs seleccionados. Es perfectamente factible que dos soluciones tengan el mismo tamaño ($k = 50$) pero eficiencias radicalmente distintas (0,2 frente a 0,5) si una de ellas ha seleccionado SNPs altamente polimórficos y la otra SNPs redundantes. La división no duplica información, sino que cambia la pregunta que el motor debe responder: de “¿cuánta distancia total acumulas?” a “¿cuál es la rentabilidad informativa de los SNPs que has elegido?”.

2. La necesidad algorítmica: El paisaje de aptitud (Fitness Landscape)

En un marco teórico con recursos computacionales infinitos, proporcionar al algoritmo los objetivos crudos permitiría que el frente de Pareto perfecto emergiera de forma natural. Sin embargo, en la práctica clínica nos enfrentamos a un espacio combinatorio inmenso (p. ej., 2^{556} posibles combinaciones) que debe explorarse con presupuestos computacionales estrictos (p. ej., poblaciones de 100 individuos durante 200 generaciones).

Bajo estas restricciones físicas, la topología del paisaje de aptitud (*fitness landscape*) es crítica. Como hemos demostrado empíricamente, si los objetivos se mantienen “puros” (Modo Absoluto, con $f_1 = k$

y $f_3 = \bar{D}(S)$), el paisaje de f_3 actúa como un sumidero gravitatorio. La facilidad matemática para ganar distancia bruta arrastra al algoritmo hacia soluciones de $k = 500$ en las primeras generaciones. El motor quema su presupuesto explorando una región del espacio hiper-saturada y biológicamente inútil, fracasando en el descubrimiento de las regiones compactas que son clínicamente viables.

Conclusión

La división propuesta por Ting no es un error de diseño, sino una **reformulación del paisaje de aptitud**. Lejos de crear redundancia, actúa como un ancla pragmática que impide que el algoritmo malgaste su presupuesto computacional en la fuerza bruta, obligándole a navegar de forma productiva hacia las soluciones de alta densidad informativa.

2.4. Desglose de las Fórmulas (Paper vs. Código de Ting)

Contrariamente a lo que sugeriste en la última reunión, analizando las fórmulas del paper y su implementación en código, me atrevería a decir que son coherentes, a excepción de la división entre k en f_3 y f_4 . Al auditar la implementación original en C++ proporcionada por los autores (`Evaluation.cpp`) contra las fórmulas publicadas en su artículo, encontramos que el código es una traducción matemática rigurosa.

Como crítica preliminar a la documentación del artículo, cabe destacar una omisión importante en el texto: las fórmulas del documento no incluyen explícitamente la división por el número de SNPs seleccionados (k , representado como `f1` en el código). Sin embargo, el código fuente (líneas 117-118) demuestra que aplican esta normalización de forma estricta. Obviando esta omisión documental que ya conocemos y asumiendo la normalización entre k , analizamos a continuación la fidelidad del resto de la implementación.

1. Evaluación de la Distancia Media (f_3)

La fórmula teórica propuesta en el artículo para calcular la distancia de Hamming promedio es:

$$f_3 = \frac{1}{\binom{n}{2}} \sum_{i < j} D(h_i, h_j)$$

En el código fuente, Ting implementa esta sumatoria mediante un acumulador simple dentro de un bucle que recorre todos los pares de haplotipos:

```
1 f3 += ham_dist[idx_count];
```

Posteriormente, normaliza este sumatorio dividiéndolo entre la variable `raw_hap_pairs`, que corresponde matemáticamente a $\binom{n}{2}$ (el número total de parejas posibles). Por lo tanto, obviando la normalización por k , el código implementa literal y perfectamente la ecuación de la media poblacional descrita en el paper.

2. Evaluación de la Varianza (f_4)

La ecuación teórica de la varianza descrita en el artículo es:

$$f_4 = \frac{1}{\binom{n}{2}} \sum_{i < j} (D(h_i, h_j) - f_3)^2$$

Esta expresión clásica obliga computacionalmente a realizar dos pasadas completas sobre los datos: una primera iteración para calcular la media (f_3), y una segunda iteración para calcular el sumatorio de las desviaciones al cuadrado. Para evitar esta ineficiencia algorítmica, los autores utilizaron astutamente la equivalencia matemática del momento estadístico: $Var(X) = E[X^2] - (E[X])^2$.

En su código, Ting crea un acumulador paralelo para los cuadrados de las distancias durante la primera y única iteración:

```
1 f4 += ham_dist[idx_count]*ham_dist[idx_count];
```

Y al final de la ejecución, calculan la varianza final restando el cuadrado de la media al promedio de los cuadrados:

```
1 f4 = (f4) / raw_hap_pairs - f3 * f3;
```

(Nota: Se ha omitido la división por k de la instrucción original en C++ para mayor claridad).

Conclusión

El análisis del código fuente confirma que los autores implementaron las métricas de forma computacionalmente inteligente. La transformación de la fórmula canónica de la varianza a su equivalencia algebraica ($E[X^2] - \mu^2$) demuestra que el código de Ting es fiel a las matemáticas de su propio paper.